

Relazione di Progetto: Sistema di Gestione Interventi di Manutenzione Condominiale

1. Informazioni Generali

Il presente documento descrive l'architettura, le scelte implementative e le modalità d'uso del programma per la gestione degli interventi di manutenzione in un condominio. Il sistema è stato sviluppato in linguaggio C ed è predisposto per la compilazione tramite Makefile in ambiente Windows.

Gruppo di lavoro: Joshua Sarnelli, Raffaele Pio Villani, Sabato Pio Siano.

Introduzione e Obiettivi del Progetto

Il progetto consiste in un sistema software gestionale ideato per automatizzare e ottimizzare l'amministrazione degli interventi di manutenzione all'interno di un condominio. L'obiettivo primario è fornire uno strumento in grado di governare l'intero ciclo di vita di una problematica: dalla sua segnalazione iniziale, fino alla risoluzione definitiva, garantendo un flusso di lavoro ordinato e tracciabile.

Dominio Applicativo e Contesto

Il contesto di riferimento è quello della gestione di un condominio. In questo scenario, le criticità (come guasti elettrici, perdite idrauliche o danni alle aree comuni) devono essere registrate tempestivamente e gestite in base alla loro gravità. Il sistema si pone come intermediario logico tra le necessità dell'edificio e la forza lavoro (i manutentori), assicurando che le emergenze vengano trattate con priorità e che le risorse umane siano impiegate in modo coerente ed efficiente.

Entità Fondamentali del Sistema

Le logiche di business del gestionale ruotano attorno a due concetti chiave che interagiscono continuamente:

- **Le Richieste di Intervento:** Rappresentano le segnalazioni vere e proprie. Ogni richiesta è caratterizzata da attributi che ne definiscono la natura (tipologia del problema, area interessata), un livello di urgenza per stabilirne le priorità e un indicatore del suo stato di avanzamento.
- **I Tecnici:** Costituiscono il personale specializzato incaricato di risolvere i guasti. Ogni tecnico è definito da competenze specifiche e possiede una propria disponibilità in agenda che il sistema deve rispettare per evitare sovrapposizioni lavorative.

Flusso di Lavoro e Logiche Operative

A livello concettuale, il sistema modella la realtà operativa garantendo le seguenti

macro-funzionalità:

1. **Tracciabilità e Avanzamento:** Le segnalazioni seguono un percorso rigoroso, evolvendo attraverso stati logici sequenziali: dall'apertura iniziale, alla pianificazione temporale, passando per la lavorazione effettiva fino alla sua conclusione o eventuale annullamento.
2. **Assegnazione Intelligente e Pianificazione:** Il sistema supporta l'abbinamento tra il guasto e il manutentore basandosi strettamente sulla compatibilità delle specializzazioni. Inoltre, isola automaticamente i ticket più critici e assicura che gli appuntamenti vengano fissati senza creare conflitti di orario per gli operatori.
3. **Monitoraggio e Reportistica Statistica:** L'applicazione aggrega i dati correnti e storici per produrre misurazioni utili all'amministrazione. Questo include il calcolo dei tempi medi necessari a chiudere una pratica, l'individuazione dei tecnici con il maggior carico di lavoro e la mappatura delle aree condominiali maggiormente soggette a usura.

2. Motivazione e Scelta delle Strutture Dati Astratte (ADT)

La corretta selezione delle Strutture Dati Astratte (ADT) rappresenta il cuore dell'architettura del software. Le scelte implementative sono state guidate dalla necessità di bilanciare il consumo di memoria (complessità spaziale) e la velocità di esecuzione (complessità temporale), rispettando i principi di modularità e *Information Hiding*. Di seguito vengono analizzate in modo approfondito ed esaustivo le motivazioni algoritmiche alla base di ciascuna ADT adottata nel progetto.

2.1 Archivio Storico delle Richieste

- **Struttura Dati Implementata:** Lista Doppia Concatenata (dinamica).
- **Contesto e Ruolo:** Memorizzare in modo persistente l'intero ciclo di vita di tutte le segnalazioni condominiali (aperte, pianificate, in lavorazione, concluse o annullate).
- **Perché è stata scelta:**

Il volume delle richieste di intervento è destinato a crescere indefinitamente nel tempo, rendendo impossibile prevederne a monte la quantità totale. Se si fosse scelto un array dinamico, il sistema avrebbe subito pesanti rallentamenti a causa delle frequenti e costose operazioni di riallocazione della memoria (*realloc*) e copia dei dati, oltre a generare frammentazione.

La **lista doppiamente concatenata** risolve questo problema allocando memoria dinamicamente solo quando serve (nodo per nodo). L'inserimento di un nuovo ticket avviene sempre in coda, garantendo un costo temporale costante **$O(1)$** grazie al mantenimento del puntatore all'ultimo elemento. Inoltre, i requisiti logici del gestionale (come la generazione di statistiche e la stampa dello storico) impongono *comunque* una scansione completa del database, operazione che si esegue in tempo lineare **$O(n)$** . Pertanto, l'assenza di accesso diretto (tipico degli array) non rappresenta un collo di bottiglia, rendendo la lista la soluzione ingegneristica più efficiente per i log continui. Il doppio collegamento (prev e next) agevola enormemente le operazioni di attraversamento bidirezionale e la rimozione fisica sicura dei nodi.

2.2 Gestore delle Priorità (Coda di Attesa)

- **Struttura Dati Implementata:** Max-Heap (implementato tramite Array Dinamico).
- **Contesto e Ruolo:** Isolare e ordinare le richieste nello stato "APERTA", in attesa di essere elaborate, per garantire che l'assegnazione avvenga per *urgenza* e non per ordine di arrivo.
- **Perché è stata scelta:**

Un sistema di manutenzione impone che le emergenze critiche vengano servite sempre prima dei guasti ordinari (politica di scheduling basata su priorità numerica da 0 a 4). L'utilizzo di una semplice lista ordinata avrebbe comportato costi inaccettabili per l'inserimento $O(n)$, mentre una lista non ordinata avrebbe richiesto $O(n)$ per ogni ricerca del massimo. Il **Max-Heap** rappresenta la sintesi algoritmica perfetta: la richiesta più urgente è sempre garantita alla radice dell'albero, consentendo una lettura immediata in $O(1)$. Quando l'emergenza viene assegnata ed estratta, il ripristino delle priorità (operazione di *heapify-down*) si completa nell'ottimo tempo logaritmico $O(\log n)$, così come l'inserimento di nuovi ticket (*heapify-up*).

2.3 Anagrafica delle Risorse (Albero dei Tecnici)

- **Struttura Dati Implementata:** Albero Binario di Ricerca (BST).
- **Contesto e Ruolo:** Gestire il personale manutentore del condominio, permettendo la registrazione, l'aggiornamento e il reperimento rapido dei profili professionali.
- **Perché è stata scelta:**

Durante l'assegnazione dei ticket, il sistema deve continuamente cercare il tecnico più adatto in base al codice identificativo univoco o alla specializzazione. In un database lineare come una lista, questa ricerca ripetitiva avrebbe una complessità inefficiente $O(n)$. L'impiego di un **Albero Binario di Ricerca (BST)**, in cui la chiave di ordinamento è il codice del manutentore, riduce drasticamente i costi computazionali. Le operazioni di inserimento di un nuovo dipendente e, soprattutto, di interrogazione per la verifica di identità avvengono in un tempo logaritmico $O(\log n)$ nel caso medio. A questo si aggiunge un formidabile vantaggio di presentazione: attraversando l'albero con una semplice *visita in-order*, il sistema è in grado di stampare a schermo l'elenco dei tecnici già perfettamente formattato in ordine alfabetico/lessicografico in $O(n)$, senza dover applicare costosi algoritmi di *sorting* esterni.

2.4 Pianificazione Appuntamenti (Agenda del Tecnico)

- **Struttura Dati Implementata:** Albero Binario di Ricerca (BST) innestato all'interno di ciascun nodo Tecnico.
- **Contesto e Ruolo:** Mantenere traccia degli incarichi assegnati al singolo manutentore per gestire e prevenire matematicamente accavallamenti orari o conflitti nell'agenda.
- **Perché è stata scelta:**

Questo è uno dei requisiti di dominio più rigidi: il software *deve assolutamente* impedire l'incrocio degli orari. Se l'agenda di un tecnico fosse un array o una lista, ogni nuovo

appuntamento richiederebbe una scansione lineare globale per validare le date, cercare eventuali collisioni orarie e, infine, inserire l'evento al giusto posto cronologico $O(n)$.

Per garantire scalabilità, l'agenda è stata modellata internamente come un **BST separato**, in cui il criterio di ordinamento è la stringa temporale (Data + Ora di inizio). Quando il sistema tenta di inserire un nuovo intervento, percorre l'albero: se rileva una sovrapposizione temporale o un nodo già esistente nello stesso slot orario, l'operazione viene immediatamente rifiutata; altrimenti, lo inserisce come foglia nella corretta posizione temporale in tempo $O(\log n)$. Questa struttura intrinsecamente auto-ordinante assicura che la visualizzazione cronologica degli impegni giornalieri (il calendario del tecnico) sia sempre immediata e corretta tramite la consueta visita *in-order*.

3. Progettazione e Architettura

Il sistema adotta uno stile architetturale modulare a strati. La scomposizione del software è guidata dal principio dell'incapsulamento e dell'information hiding, separando la gestione dell'interfaccia utente, la logica applicativa e le strutture dati deputate alla memorizzazione e manipolazione delle informazioni.

3.1 Scomposizione in Moduli

L'architettura del codice sorgente è organizzata in tre macro-livelli logici:

- **Livello di Presentazione e Controllo (main.c, utilita.c):** Gestisce l'interfaccia a console, il menu interattivo e il coordinamento delle operazioni principali in risposta agli input dell'utente.
- **Livello delle Strutture Dati Astratte e Logica (archivioRichieste.c, codaPriorita.c, alberoTecnici.c, agendaTecnico.c, report.c):** Implementa i motori algoritmici del sistema esponendo le primitive di manipolazione per le ADT.
- **Livello delle Entità Base (richiesta.c, tecnico.c):** Definisce i tipi di dato strutturati fondamentali e i relativi metodi di astrazione (costruttori, distruttori, getter/setter).

3.2 Interazione tra i Moduli e Flussi Logici

- **Flusso di Registrazione di una Segnalazione:** Il modulo main invoca il costruttore in richiesta.c, poi invia il puntatore a archivioRichieste.c (lista concatenata) e codaPriorita.c (Max-Heap).
- **Flusso di Assegnazione e Risoluzione Conflitti:** Il main estrae la richiesta da codaPriorita.c, interroga alberoTecnici.c per trovare il tecnico, e agendaTecnico.c verifica le sovrapposizioni orarie prima di validare l'appuntamento.
- **Flusso di Rimozione Logica:** Per evitare costose riorganizzazioni, richiesta.c e codaPriorita.c usano un flag logico (isValidInHeap) per la lazy deletion.

4. Come usare e interagire con il sistema

Il sistema offre un'interfaccia a riga di comando fluida e un menu testuale interattivo che

nasconde la complessità delle strutture dati sottostanti.

1. **Compilazione e avvio:** Il progetto usa un Makefile. Aprire il terminale e digitare make, quindi avviare l'eseguibile generato.
2. **Navigazione del menu:** L'utente digita il numero corrispondente all'azione desiderata (con controlli anti-errore sugli input).
3. **Operazioni di base:**
 - **Inserimento segnalazione:** Guidato passo dopo passo (descrizione, area, urgenza).
 - **Gestione tecnici:** Registrazione o visualizzazione della lista in ordine alfabetico.
 - **Assegnazione intervento:** Proposta automatica dell'urgenza massima; controllo automatico dell'agenda del tecnico per evitare sovrapposizioni.
 - **Consultazione e reportistica:** Controllo impegni cronologici o stampe riepilogative generali.
4. **Chiusura sicura:** Terminazione pulita che svuota e dealloca tutta la memoria utilizzata, prevenendo memory leak.

5. Razionale dei Casi di Test

La suite di collaudo automatizzata si basa sull'uso rigoroso di asserzioni (assert) e sul confronto sistematico dell'output con file "Oracolo" per garantire l'Information Hiding.

- **Test 1:** Registrazione e Ordinamento Richieste (Verifica invarianza del Max-Heap).
- **Test 2:** Popolamento e Visita Anagrafica Tecnici (Verifica BST in-order).
- **Test 3:** Assegnazione Corretta (Verifica accoppiamento richiesta-tecnico).
- **Test 4:** Pianificazione e Prevenzione Conflitti (Blocco sovrapposizioni orarie).
- **Test 5:** Transizioni di Stato (Ciclo di vita della richiesta).
- **Test 6:** Ricerca e Filtri (Ricerca globale senza crash su miss).
- **Test 7:** Estrazione dello Storico Interventi (Visita lista per auditing).
- **Test 8:** Statistiche e Reportistica Aggregata (Conteggi precisi).
- **Test 9:** Stress Test Multi-Slot Stesso Giorno (Pianificazione ad alta granularità temporale).

6. Specifica Sintattica e Semantica (incluse funzioni interne .c)

Le specifiche delle funzioni sono state organizzate in tabelle dettagliate per facilitarne la consultazione, includendo sia l'interfaccia pubblica che le funzioni di utilità interne (static) definite nei file sorgente.

6.1 Modulo: richiesta.c / richiesta.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
creaRichiesta	Alloca e inizializza una	Parametri stringa non	Nuova Richiesta allocata in heap	Effetti: Alloca memoria

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	nuova richiesta di manutenzione. Stato iniziale APERTA, flag validità a 1.	NULL. Codice univoco. Data valida.	con campi opzionali a NULL.	dinamica (malloc/strdup). Ritorna: Puntatore alla Richiesta o NULL.
distruggiRichiesta	Libera tutta la memoria associata a una Richiesta, inclusi campi stringa.	Target allocato != NULL.	Memoria liberata, puntatore invalidato.	Effetti: Dealloca heap (free). Ritorna: void.
getCodiceRichiesta	Restituisce il codice univoco della richiesta.	Nessuna (gestisce target NULL).	Nessuna modifica.	Ritorna: Stringa codice o NULL. Operazione di sola lettura.
getAppartamentoRichiesta	Restituisce il codice dell'appartamento.	Nessuna (gestisce target NULL).	Nessuna modifica.	Ritorna: Stringa appartamento o NULL.
getTipologiaProblemaRichiesta	Restituisce la categoria del problema segnalato.	Nessuna.	Nessuna modifica.	Ritorna: Stringa tipologia o NULL.
getDescrizioneRichiesta	Restituisce la descrizione dettagliata.	Nessuna.	Nessuna modifica.	Ritorna: Stringa descrizione o NULL.
getDataRichiesta	Restituisce la data di apertura (GG/MM/AAAA).	Nessuna.	Nessuna modifica.	Ritorna: Stringa data o NULL.
getLivelloUrgenzaRichiesta	Restituisce l'urgenza numerica per la	Nessuna.	Nessuna modifica.	Ritorna: Intero urgenza o -1 se NULL.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	priorità.			
getStatoRichiesta	Restituisce lo stato corrente.	Nessuna.	Nessuna modifica.	Ritorna: Enum StatoRichiesta o ANNULLATA se NULL.
getCodiceTecnicoAssegnatoRichiesta	Restituisce il codice del tecnico assegnato.	Nessuna.	Nessuna modifica.	Ritorna: Stringa codice o NULL se non assegnato.
getDataInizioLavorazioneRichiesta	Restituisce data inizio lavorazione.	Nessuna.	Nessuna modifica.	Ritorna: Stringa data o NULL.
getDataChiusuraRichiesta	Restituisce data completamento.	Nessuna.	Nessuna modifica.	Ritorna: Stringa data o NULL.
getFasciaOrariaRichiesta	Restituisce fascia oraria (HH:MM-HH:MM).	Nessuna.	Nessuna modifica.	Ritorna: Stringa orario o NULL.
isValidaInHeapRichiesta	Indica se la richiesta è attiva (lazy deletion).	Nessuna.	Nessuna modifica.	Ritorna: 1 se valida, 0 altrimenti.
setStatoRichiesta	Aggiorna lo stato nel ciclo di vita.	Target valido o NULL.	Stato aggiornato.	Modifica campo stato dell'oggetto. Ritorna: void.
setCodiceTecnicoAssegnatoRichiesta	Assegna/riassegna il codice di un tecnico.	Target valido o NULL.	Codice precedente liberato e sostituito con strdup.	Effetti: free e strdup. Ritorna: void.
setDataInizioLavorazione	Imposta data	Target valido o NULL.	Data aggiornata	Effetti: free e strdup.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
orazioneRichiesta	inizio lavori.	NULL.	tramite strdup.	strdup. Ritorna: void.
setDataChiusuraRichiesta	Imposta data di chiusura.	Target valido o NULL.	Data aggiornata tramite strdup.	Effetti: free e strdup. Ritorna: void.
setValidaInHeapRichiesta	Aggiorna flag validità per coda priorità.	Target valido o NULL.	Flag aggiornato (se 0 scartato logicamente).	Nessuna allocazione. Ritorna: void.
setFasciaOrariaRichiesta	Imposta fascia pianificata.	Target valido, fascia in formato standard.	Fascia liberata e sostituita.	Effetti: free e strdup. Ritorna: void.

6.2 Modulo: tecnico.c / tecnico.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
creaTecnico	Alloca e inizializza un nuovo tecnico con agenda vuota.	Stringhe non NULL e valide. Codice univoco.	Tecnico allocato in heap, disponibile(1).	Effetti: malloc/strdup, crea agenda. Ritorna: Puntatore a Tecnico.
distruggiTecnico	Libera memoria associata al Tecnico, agenda inclusa.	Target allocato != NULL.	Memoria liberata, puntatore invalidato.	Ritorna: void. Dealloca heap (free).
getCodiceTecnico	Restituisce codice univoco.	Nessuna.	Nessuna modifica.	Ritorna: Stringa codice o NULL.
getNomeTecnico	Restituisce nome completo.	Nessuna.	Nessuna modifica.	Ritorna: Stringa nome o NULL.
getSpecializzazioni	Restituisce	Nessuna.	Nessuna	Ritorna: Stringa

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
oneTecnico	specializzazione principale.		modifica.	specializzazione o NULL.
isDisponibileTecnico	Verifica se il tecnico accetta incarichi.	Nessuna.	Nessuna modifica.	Ritorna: 1 se disponibile, 0 altrimenti.
getAgendaTecnico	Restituisce puntatore all'agenda interventi.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore AgendaTecnico o NULL.
setDisponibilitaTecnico	Aggiorna flag di disponibilità.	Target valido o NULL.	Flag aggiornato.	Ritorna: void.
setSpecializzazioneTecnico	Modifica specializzazione del tecnico.	Target valido o NULL.	Specializzazione e aggiornata tramite strdup.	Effetti: free e strdup. Ritorna: void.

6.3 Modulo: utilita.c / utilita.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
duplicaStringa	Alloca in heap una copia della stringa. Compatibile con C89/C99.	Nessuna (gestisce NULL).	Buffer heap allocato con copia.	Effetti: malloc. Ritorna: Puntatore copia o NULL.
validaData	Verifica sintassi e logica data (GG/MM/AAAA).	Nessuna.	Nessuna modifica.	Ritorna: 1 se valida, 0 altrimenti.
validaDataChiusuraRichiesta	Valida data chiusura verificando coerenza ($\geq$ apertura/inizio).	Richiesta valida.	Nessuna modifica.	Ritorna: 1 se coerente, 0 altrimenti.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
confrontaDate (static)	Confronta due date per ordinamento cronologico.	Nessuna (gestisce NULL).	Nessuna modifica.	Ritorna: 1 se data1 precede, -1 se segue, 0 se uguali.
pulisciBuffer	Svuota buffer stdin fino a newline/EOF.	Nessuna.	Buffer stdin vuoto.	Consuma caratteri pendenti. Ritorna: void.
pulisciSchermo	Cancella contenuto terminale (cls/clear).	Nessuna.	Terminale visivamente pulito.	Effetti: chiamata di sistema. Ritorna: void.
pausaSchermo	Sospende esecuzione attendendo INVIO.	Nessuna.	Ripresa dopo INVIO.	Effetti: Stampa e blocco in attesa. Ritorna: void.
acquisisciStringa	Stampa prompt e legge riga da stdin, rimuovendo newline.	Prompt e buffer non NULL, dim > 0.	Buffer contiene stringa inserita.	Modifica array buffer. Ritorna: void.
statoRichiestaToString	Converte enum StatoRichiesta in stringa italiana.	Stato valido enum.	Nessuna modifica.	Ritorna: Stringa costante o "SCONOSCIUTO".
stampaTecnico	Stampa riga tabellare con dati tecnico.	Nessuna (gestisce NULL).	Output su stdout.	Ritorna: void.
stampaRichiesta	Stampa riga tabellare con dati richiesta.	Nessuna (gestisce NULL).	Output su stdout.	Ritorna: void.
trovaTecnicoDis	Visita ricorsiva	Nessuna.	Nessuna	Ritorna:

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
ponibilePerSpecializzazioneRicerca (static)	in-order dell'albero per trovare tecnico compatibile.		modifica al BST.	Puntatore Tecnico trovato o NULL.
trovaTecnicoDisponibilePerSpecializzazione	Cerca in BST il primo tecnico disponibile per specializzazione.	Nessuna.	Nessuna modifica al BST.	Wrapper pubblico. Ritorna: Puntatore Tecnico o NULL.
pianificaIntervento	Pianifica intervento inserendolo in agenda e aggiornando stato.	Nessuna. (Richiede tecnico/richesta validi).	Richiesta ASSEGNATA, agenda aggiornata.	Effetti: malloc nodo agenda. Ritorna: 1 successo, 0 errore.
stampaStoricoInterventi	Stampa storico richieste concluse.	Nessuna.	Output su stdout.	Ritorna: void.
stampaReportStatistiche	Genera report sintetico di sistema.	Nessuna.	Output su stdout.	Ritorna: void.
visitaAgendaInOrder (static)	Visita in-order dell'agenda per stampa tabellare numerata.	Puntatore a contatore (numero) valido.	Stampa nodi, contatore incrementato.	Ritorna: void.
stampaAgendaTecnico	Stampa agenda appuntamenti cronologici di un tecnico.	Nessuna.	Output su stdout.	Ritorna: void.
stampaCaricoLavoroTecnico	Stampa carico lavoro colorato in base a saturazione.	Nessuna.	Output su stdout.	Effetti: Escape ANSI per colori. Ritorna: void.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
caricaTecniciDaFile	Legge file testo e popola albero tecnici.	Albero valido. File leggibile.	Albero modificato con nuovi nodi.	Effetti: I/O file, malloc. Ritorna: Numero caricati.
caricaRichiesteDaFile	Legge file e popola archivio e coda priorità.	Strutture inizializzate. File leggibile.	Archivio e coda aggiornati.	Effetti: I/O file, malloc. Ritorna: Numero caricate.
confrontaFileOracolo	Confronta output con oracolo per testing.	Nessuna.	Nessuna modifica.	Effetti: I/O disco. Ritorna: 1 identici, 0 se differiscono, -1 errore.
validaFasciaOra	Verifica formato e logica (HH:MM-HH:MM, range 0-23/0-59).	Nessuna.	Nessuna modifica.	Ritorna: 1 se valida, 0 altrimenti.

6.4 Modulo: agendaTecnico.c / agendaTecnico.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
confrontaAppuntamenti (static)	Confronta cronologicamente e matematicamente date e fasce orarie.	Stringhe data e ora valide.	Nessuna modifica.	Ritorna: <0 se precede, >0 se segue, 0 se sovrappongono.
inserisciNodoRicorsivo (static)	Inserisce nodo nell'albero in ordine cronologico.	Dati intervento validi.	Nodo allocato se non vi sono conflitti.	Effetti: malloc. Ritorna: Radice sottoalbero aggiornata.
distruggiNodiRicorsivo (static)	Dealloca nodi sottoalbero con	Nessuna.	Memoria nodi e stringhe interne	Ritorna: void.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	attraversamento post-order.		deallocata.	
eliminaNodoRicorsivo (static)	Rimuove nodo corrispondente a data/fascia e ribilancia (BST).	Nessuna.	Nodo deallocato, albero ricongiunto.	Ritorna: Radice sottoalbero aggiornata.
contaNodiAgenda (static)	Conta ricorsivamente i nodi del BST dell'agenda.	Nessuna.	Nessuna modifica.	Ritorna: Intero totale nodi.
creaAgendaTecnico	Alloca/inizializza agenda vuota per tracciare interventi (come BST).	Nessuna.	Struttura allocata pronta.	Effetti: malloc. Ritorna: Puntatore AgendaTecnico.
distruggiAgendaTecnico	Libera memoria agenda, inclusi nodi e stringhe.	Target allocato != NULL.	Memoria liberata (albero e stringhe).	Ritorna: void.
inserisciInterventoInAgenda	Inserisce nuovo appuntamento mantenendo ordine cronologico. Verifica sovrapposizioni.	Puntatori stringhe non NULL, data/fascia valide.	Nodo aggiunto al BST, se no conflitti.	Ritorna: 1 successo, 0 errore/conflitto.
getNumeroInterventiAgenda	Conta appuntamenti pianificati.	Nessuna.	Nessuna modifica.	Ritorna: Intero.
getRadiceAgenda	Restituisce radice BST agenda.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAgenda o NULL.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
getFiglioSinistroAgenda	Restituisce sottoalbero sinistro (interventi precedenti).	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAgenda o NULL.
getFiglioDestroAgenda	Restituisce sottoalbero destro (interventi successivi).	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAgenda o NULL.
getDataIntervento	Restituisce data appuntamento.	Nessuna.	Nessuna modifica.	Ritorna: Stringa data o NULL.
getFasciaOrariaIntervento	Restituisce fascia oraria.	Nessuna.	Nessuna modifica.	Ritorna: Stringa fascia o NULL.
getCodiceRichiestaInAgenda	Restituisce codice richiesta associata.	Nessuna.	Nessuna modifica.	Ritorna: Stringa codice o NULL.
rimuoviInterventoDaAgenda	Rimuove nodo da BST agenda per cancellazioni (mantiene ordinamento).	Data/fascia identificano nodo univocamente.	Nodo e relative stringhe deallocate.	Ritorna: void.

6.5 Modulo: `alberoTecnici.c` / `alberoTecnici.h`

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
inserisciNodoRicorsivo (static)	Inserimento nel BST rispettando ordine lessicografico del codice.	Nessuna (gestisce NULL).	Nodo aggiunto se non duplicato.	Ritorna: Radice aggiornata.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
cercaTecnicoRicorsivo (static)	Ricerca binaria ($O(\log N)$) per codice tecnico.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore Tecnico trovato o NULL.
distruggiNodiRicorsivo (static)	Attraversamento post-order per deallocare nodi e Tecnici.	Nessuna.	Memoria sottoalbero liberata.	Ritorna: void.
visitaRicorsivo (static)	Attraversamento in-order per applicare visitatore ai Tecnici.	Nessuna.	Visitatore applicato in ordine.	Ritorna: void.
creaAlberoTecnici	Alloca/inizializza BST vuoto per gestione tecnici.	Nessuna.	Struttura allocata (radice NULL).	Effetti: malloc. Ritorna: Puntatore AlberoTecnici.
distruggiAlberoTecnici	Libera memoria albero e tecnici contenuti.	Target allocato != NULL.	Memoria liberata in profondità.	Ritorna: void.
inserisciInAlberoTecnici	Inserisce tecnico nell'albero. Ignora duplicati.	Target e Tecnico validi/allocati.	Nuovo nodo allocato nel BST.	Ritorna: void.
cercaTecnicoInAlbero	Ricerca binaria ($O(\log n)$) per codice identificativo.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore Tecnico o NULL.
getRadiceAlberoTecnici	Restituisce radice BST.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAlberoTecnici o NULL.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
getFiglioSinistroTecnici	Restituisce tecnico codice minore.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAlberoTecnici o NULL.
getFiglioDestroTecnici	Restituisce tecnico codice maggiore.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoAlberoTecnici o NULL.
getTecnicoDalNodoAlbero	Estrae payload Tecnico dal nodo.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore Tecnico o NULL.
visitaAlberoTecnici	Visita in-order applicando funzione visitatore a tutti i Tecnici.	Nessuna.	Funzione visitatore applicata in ordine alfabetico.	Effetti: quelli del visitatore. Ritorna: void.

6.6 Modulo: `archivioRichieste.c` / `archivioRichieste.h`

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
creaArchivioRichieste	Alloca archivio basato su lista doppiamente concatenata.	Nessuna.	Struttura allocata, pronta all'uso.	Effetti: malloc. Ritorna: Puntatore ArchivioRichieste.
distruggiArchivioRichieste	Libera archivio e iterativamente i suoi nodi e richieste.	Target allocato != NULL.	Tutta la memoria liberata.	Ritorna: void.
inserisciInCodaArchivio	Accoda richiesta alla lista doppiamente concatenata.	Target e Richiesta validi.	Richiesta accodata, dimensione incrementata.	Effetti: malloc nodo. Ritorna: void.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
rimuoviNodoDaArchivio	Rimuove fisicamente nodo e lo dealloca, ripristinando next/prev.	Puntatori validi, nodo appartiene all'archivio.	Nodo rimosso, memoria liberata.	Ritorna: void.
stampaRichiesteArchivioPerStato	Filtra lista archivio per Stato e stampa le corrispondenze.	Stato valido.	Output su stdout.	Ritorna: void.
getTestaArchivio	Restituisce primo nodo della lista.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoLista o NULL.
getCodaArchivio	Restituisce ultimo nodo lista.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoLista o NULL.
getNextNodoLista	Restituisce nodo successivo.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoLista o NULL.
getPrevNodoLista	Restituisce nodo precedente.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore NodoLista o NULL.
getRichiestaDalNodoLista	Estrae payload Richiesta dal nodo lista.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore Richiesta o NULL.
getDimensioneArchivio	Restituisce dimensione	Nessuna.	Nessuna modifica.	Ritorna: Intero contatore.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	lista.			
cercaRichiestaPerCodice	Ricerca lineare ($O(N)$) in archivio per codice univoco.	Nessuna.	Nessuna modifica.	Ritorna: Puntatore Richiesta o NULL.
stampaRichiesteArchivioPerUrgenza	Filtra lista per livello di urgenza.	Urgenza valida (0-4 raccomandato).	Output su stdout.	Ritorna: void.
stampaRichiesteArchivioPerTipologia	Filtra per tipologia esatta (match stringa).	Nessuna.	Output su stdout.	Ritorna: void.
stampaRichiesteArchivioPerAppartamento	Filtra per appartamento o area di provenienza.	Nessuna.	Output su stdout.	Ritorna: void.
stampaRichiesteArchivioPerTecnico	Filtra storico per codice tecnico assegnato.	Nessuna.	Output su stdout.	Ritorna: void.

6.7 Modulo: codaPriorita.c / codaPriorita.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
scambia (static)	Scambia due puntatori a Richiesta nell'array dinamico.	Indirizzi validi.	Posizioni invertite.	Ritorna: void.
heapifyUp (static)	Ripristina max-heap dal basso (urgenza maggiore verso	Coda/indice validi.	Sottoalbero bilanciato.	Ritorna: void. Ricorsiva.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	l'alto).			
heapifyDown (static)	Ripristina max-heap dall'alto (sprofonda nodi meno urgenti).	Coda/indice validi.	Sottoalbero bilanciato.	Ritorna: void. Ricorsiva.
creaCodaPriorita	Inizializza Max-Heap basato su array dinamico.	Capacità iniziale > 0.	Struttura allocata per inserimenti $O(\log N)$.	Effetti: malloc. Ritorna: Puntatore CodaPriorita.
distruggiCodaPriorita	Libera coda e array interno.	Target allocato.	Memoria base e array liberati.	Non dealloca le Richieste (responsabilità Archivio). Ritorna: void.
inserisciInCodaPriorita	Inserisce richiesta e ripristina Max-Heap (heapify-up).	Target/Richiesta validi.	Inserito, possibile realloc. Heap mantenuto.	Ritorna: void.
estraiMaxDaCodaPriorita	Estrae root più urgente (heapify-down). Supporta lazy deletion.	Nessuna.	Rimuove massima, filtra invisibilmente eliminate logicamente.	Ritorna: Puntatore Richiesta (più urgente) o NULL.
isCodaPrioritaVuota	Verifica se heap è vuoto/tutti rimossi logicamente.	Nessuna.	Nessuna modifica.	Ritorna: 1 se vuota, 0 altrimenti.
getDimensioneCodaPriorita	Ottiene contatore globale nodi (inclusi	Nessuna.	Nessuna modifica.	Ritorna: Intero.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	invalidati).			
stampaCodaPriorita	Stampa completo in ordine di priorità.	Nessuna.	Output su stdout.	Ritorna: void.
stampaRichiestePerStato	Stampa richieste in heap filtrate per Stato.	Stato valido.	Output su stdout.	Ritorna: void.
stampaRichiestePerUrgenza	Stampa richieste in heap filtrate per urgenza.	Urgenza valida.	Output su stdout.	Ritorna: void.
stampaRichiestePerTipologia	Stampa richieste in heap filtrate per categoria.	Nessuna.	Output su stdout.	Ritorna: void.
stampaRichiestePerAppartamento	Stampa richieste in heap filtrate per area.	Nessuna.	Output su stdout.	Ritorna: void.
stampaRichiestePerTecnico	Stampa richieste in heap assegnate a tecnico.	Nessuna.	Output su stdout.	Ritorna: void.

6.8 Modulo: report.c / report.h

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
dataAGiorni (static)	Converte data (GG/MM/AAAA) in numero assoluto di	Nessuna.	Nessuna modifica.	Ritorna: Intero (giorni). Gestisce anni bisestili.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	giorni dal 1900.			
differenzaGiorni (static)	Calcola distanza temporale tra due date in giorni.	Nessuna.	Nessuna modifica.	Ritorna: Intero differenza.
contaInterventiPerTipologia (static)	Ricerca lineare in archivio per match tipologia.	Nessuna.	Nessuna modifica.	Ritorna: Intero (occorrenze).
contaInterventiPerTecnico (static)	Ricerca lineare in archivio per match codice tecnico.	Nessuna.	Nessuna modifica.	Ritorna: Intero (occorrenze).
contaInterventiPerAppartamento (static)	Ricerca lineare in archivio per match appartamento.	Nessuna.	Nessuna modifica.	Ritorna: Intero (occorrenze).
contaInterventiPerStato (static)	Ricerca lineare in archivio per enum Stato.	Nessuna.	Nessuna modifica.	Ritorna: Intero (occorrenze).
raccogliTipologie (static)	Crea array dinamico raggruppando interventi per tipologie uniche.	Puntatori output validi.	Alloca e restituisce l'array.	Ritorna: void. Chiamante dealloca array.
raccogliAree (static)	Crea array dinamico raggruppando interventi per appartamenti univoci.	Puntatori output validi.	Alloca e restituisce l'array.	Ritorna: void. Chiamante dealloca array.
generaReportCompleto	Report globale strutturato	Nessuna.	Output ANSI su stdout.	Ritorna: void.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
	(statistiche, tempi, tecnici).			
generaReportPerTipologia	Raggruppa e conta interventi per categoria.	Nessuna.	Tabella riepilogativa su stdout.	Ritorna: void.
generaReportStatoInterventi	Calcola proporzioni (aperti, chiusi, % completamento).	Nessuna.	Statistiche scritte su stdout.	Ritorna: void.
generaReportTempoMedio	Calcola tempo medio in gg (chiusura - apertura).	Nessuna.	Report in giorni su stdout.	Ritorna: void.
generaReportTecnicoPiuAttivo	Analizza storico per trovare tecnico con max completamenti.	Nessuna.	Classifica scritta su stdout.	Ritorna: void.
generaReportAreeProblematiche	Aggrega conteggio per appartamento/area, ordina per criticità.	Nessuna.	Elenco decrescente aree scritto su stdout.	Ritorna: void.

7. Funzioni MAIN.C e Logiche di Controllo

Il modulo main.c rappresenta l'entry point del sistema e funge da coordinatore per le chiamate tra le ADT, oltre a gestire i controlli di validazione per le sovrapposizioni e le disponibilità dei tecnici tramite funzioni interne critiche.

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
controllaIncrocio	Verifica matematicamente	Le stringhe devono	Nessuna modifica allo	Converte gli orari in minuti

Nome	Descrizione	Pre-condizioni	Post-condizioni	Note
Orari (static)	e se due fasce orarie si sovrappongono per evitare conflitti nell'agenda strutturata dei tecnici.	rispettare rigidamente il formato "HH:MM-HH:MM".	stato del sistema; operazione di sola lettura.	assoluti. Ritorna 1 in caso di incrocio, 0 altrimenti. <i>Vedi implementazione in basso.</i>
isTecnicoStalav orando (static)	Scansiona l'archivio storico per determinare se un tecnico risulta attivamente impegnato sul campo (IN_LAVORAZIONE) in una determinata data e ora.	L'archivio delle richieste deve essere validamente inizializzato.	Sola lettura. Impedisce l'errore logico di avvio simultaneo di interventi concorrenti.	Prende in esame i ticket in stato "IN_LAVORAZIONE". Ritorna 1 se viene riscontrato un conflitto, 0 altrimenti. <i>Vedi implementazione in basso.</i>
main	Entry point del sistema. Inizializza le ADT, gestisce i comandi utente e garantisce la corretta deallocazione finale.	Disponibilità dei moduli e librerie collegate.	Deallocazione completa della memoria (heap) prima della terminazione, prevenendo memory leak.	Ritorna 0 in caso di terminazione corretta. <i>Vedi implementazione in basso.</i>